SMART EV HUB

Detailed Product Requirements Document (PRD) — Updated

EV Charging & Battery-Swap Queue Management System
2nd-Year B.Tech Project-Based Learning Project

Frontend: React + TypeScript + Vite + MUI Material 3 **Backend:** C++ + OOP + Crow **DSA:** C++ Queue + Stack
Database: SQLite

Project Repository

GitHub: https://github.com/abhighansala/SmartEvHub.git
All six team members will commit their work to this shared repository. Feature branches and pull requests are recommended; the main branch should remain buildable.

1. Executive Summary

Smart EV Hub is a web-based simulation and management system for an electric-vehicle charging and battery-swap facility. It models limited charging stations and replacement batteries, organizes waiting requests with FIFO queues, records completed operations with a LIFO transaction stack, persists information in SQLite, and exposes the C++ functionality to a React web interface through REST APIs. The current React application is an interactive prototype using mock operational data; the final stage replaces mock business logic with the C++/Crow backend without redesigning the existing UI.

2. Team & Equal Contribution

Member	Primary ownership	Special responsibility
Aditya Singh — 2519272 — ML1	Database + backend foundation	Team Lead
Priyanshu Dutt — 2519460 — ML3	EV Management	EV module owner
Mritunjay Thakur — 2519425 — ML5	Charging Stations	Station module owner
Amit Singh Kaintura — 2518188 — G	Battery + Battery Swap	Battery/swap owner
Abhishek Ghansala — 2518132 — H	UI + DSA	Final compilation/integration
Agreev Singh — 2518161 — F	Transactions + Reports + Analytics	Reporting owner

Workload rule: all six members have equal overall workload. Ownership differs by module, not by effort. Every member contributes implementation, testing, documentation and presentation. Aditya's lead role and Abhishek's integration role are coordination responsibilities, not extra workload allocation.

3. Problem Statement

An EV hub has limited charging stations and a limited inventory of replacement batteries. When demand exceeds capacity, requests must wait in an organized order. Operators also need to track EVs, stations, batteries, active sessions, completed transactions and operational statistics. Smart EV Hub models this complete workflow while demonstrating C++ OOP, DSA, database management and web development.

4. Objectives

• Demonstrate practical C++ OOP through domain classes and managers.

• Implement Queue for charging and swap waiting lists and Stack for recent/completed transactions.

• Persist operational data using SQLite.

• Expose C++ functionality through REST APIs using Crow.

• Provide a polished React Material 3 web interface.

• Create a complete end-to-end demonstration suitable for a second-year PBL.

5. Scope

Included: EV management, station management, battery inventory, charging requests, swap requests, FIFO queues, active sessions, transaction stack/history, reports, analytics, settings, simple email/password login, SQLite persistence, REST APIs and responsive UI.

Excluded: real charging hardware, IoT, GPS, payment gateways, AI/ML, cloud infrastructure, industrial charging protocols, social login, complex role-based authentication and advanced DSA such as trees/graphs/heaps.

6. Technology & Architecture

Frontend: React 18, TypeScript, Vite, MUI v6 using Material 3 principles, React Router v7 and Recharts. The existing UI contains Login, AppShell, Dashboard, EV, Station, Battery, Queue, Session, Transaction, Reports, Analytics and Settings pages.

Backend: C++ with OOP and Crow for HTTP/REST routing. Business rules, DSA and database operations belong to the backend.

Database: SQLite using a local evhub.db file.

Final data flow: React → HTTP/JSON → Crow → C++ OOP/DSA → SQLite → JSON response → React.

7. Authentication — Simple Email & Password Login

The Login page will use a simple email-and-password based authentication flow. It is not a mock-only login. User credentials will be stored in the project database and validated by the C++ backend.

Initial flow: user enters email and password → React sends credentials to the C++ login endpoint → backend checks the Users table → valid credentials return success/session information → React opens AppShell/Dashboard; invalid credentials return an error message.

Users table: user_id (PK), name, email (UNIQUE), password/password representation, created_at. For this educational project, authentication is intentionally simple. The implementation should avoid storing plaintext passwords if practical; a basic password-hashing approach may be used if the team has sufficient time, but complex authentication frameworks are out of scope.

Example endpoint: POST /api/login. The frontend should keep authentication state and protect application routes after successful login. Logout should clear the frontend session and return to Login.

The purpose is simple email/password authentication, not enterprise-grade identity management.

8. Existing UI & Frontend Working

The current frontend uses AppContext and mock operational data as an in-memory prototype. Components consume state through useApp(); actions such as Complete Session and Process Next currently update React state. During backend integration, these actions will call centralized service/API functions. Pages must not directly access SQLite or contain final business rules.

Recommended boundary: Page → AppContext/service → fetch() → Crow API → C++ logic → SQLite. This preserves the current UI while replacing only its data source and business operations.

9. UI Pages & Requirements

Login.tsx — Simple email/password login. Sends credentials to the backend in the final version and redirects valid users to the application.

AppShell.tsx — Persistent Sidebar and TopBar; navigation, notifications, search and theme controls.

Dashboard.tsx — KPIs, station status, queue previews, active sessions, recent transactions and Recharts visualizations.

EVManagement.tsx — EV CRUD, search, filtering, registration and deletion.

EVDetails.tsx — EV status, battery information, charging history and transactions.

ChargingStations.tsx — Station grid/list with Available, Charging, Maintenance and Offline states.

StationDetails.tsx — Station specifications, current EV/session and charging progress.

BatteryManagement.tsx — Battery inventory, state of charge and status/health.

BatterySwap.tsx — Initiate and complete swaps using available battery inventory.

ChargingQueue.tsx — Visual FIFO queue with FRONT/REAR and Process Next.

SwapQueue.tsx — FIFO queue for battery-swap requests.

ActiveSessions.tsx — Current charging sessions, progress, duration and completion.

Transactions.tsx — Transaction history plus visual LIFO stack.

Reports.tsx — Operational summaries such as sessions, energy, revenue, waiting time and utilization.

Analytics.tsx — Charts for peak hours, revenue, queue length, station utilization and battery usage.

Settings.tsx — Light/dark mode, notification preferences and application information.

10. Core Business Workflows

Charging: EV requests charging → backend checks station → if available, station is assigned and session starts → otherwise request enters Charging Queue → when a station is released, the front request is dequeued → session completes → transaction is generated → transaction is pushed to Stack and saved in SQLite.

Battery swap: EV requests swap → backend checks READY inventory → if available, battery is assigned → otherwise request enters Swap Queue → when available, the front request is processed → transaction is generated → database is updated.

Transaction: completed charging/swap creates a record with ID, EV, type, amount, timestamp and status. The C++ Stack demonstrates LIFO; SQLite preserves full history.

11. C++ OOP Model

Core classes: EV, Battery, ChargingStation, ChargingSession, Transaction and User. Manager/service classes may coordinate workflows where useful. Classes must encapsulate data and expose clear member functions. Avoid unnecessary advanced design patterns.

Relationships: a ChargingSession refers to an EV and Station; a swap refers to an EV and Battery; a Transaction records the completed operation; a User represents an authenticated application user. IDs connect persistent records.

12. DSA Requirements

Charging Queue: manual C++ FIFO structure with enqueue, dequeue, peek, isEmpty, isFull and display.

Swap Queue: manual C++ FIFO structure for swap requests.

Transaction Stack: manual C++ LIFO structure with push, pop, peek, isEmpty, isFull and display.

The current React prototype may use arrays to simulate visual behavior, but the final academic Queue and Stack must be implemented and executed in C++. React displays the state returned by the backend.

13. Database Design

Users: user_id (PK), name, email (UNIQUE), password/password representation, created_at.

EV: vehicle_id (PK), owner_name, model, battery_capacity, current_charge, status.

ChargingStation: station_id (PK), power, connector_type, status, current_vehicle_id.

Battery: battery_id (PK), capacity, charge_level, status, health.

ChargingSession: session_id (PK), vehicle_id, station_id, start_charge, target_charge, required_energy, charging_time, status, timestamps.

Transaction: transaction_id (PK), vehicle_id, type, description, amount, timestamp, status.

SQLite is the persistence layer. CRUD operations must be tested independently and records must remain after application restart.

14. REST API Contract

Authentication: POST /api/login, POST /api/logout (or equivalent session-clearing mechanism).

EV: GET /api/evs, GET /api/evs/:id, POST /api/evs, PUT /api/evs/:id, DELETE /api/evs/:id

Stations: GET /api/stations, POST /api/stations, PUT /api/stations/:id

Batteries: GET /api/batteries, POST /api/batteries, PUT /api/batteries/:id

Charging: POST /api/charging/request, GET /api/charging/queue, POST /api/charging/start, POST /api/charging/complete

Swap: POST /api/swap/request, GET /api/swap/queue, POST /api/swap/process

Transactions/Reports: GET /api/transactions, GET /api/reports, GET /api/analytics

All requests/responses use JSON. Centralize API calls in frontend services rather than scattering URLs through components.

15. Detailed Member Work Plans

Aditya Singh — Team Lead — Database + Backend Foundation

• Design Users and application database schema, initialization and relationships.

• Implement database connection and CRUD helpers.

• Define common C++ backend/API conventions and server structure.

• Set up Crow integration and common server configuration.

• Implement login backend/database validation for simple email/password authentication.

• Implement database-facing models/services as required by modules.

• Coordinate interfaces between modules and communicate with the mentor.

• Maintain repository standards and milestones.

• Test persistence and backend/database integration.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

Priyanshu Dutt — EV Management

• Implement EV C++ class and validation.

• Implement EV database operations and REST endpoints.

• Connect EV page operations to APIs.

• Maintain EVManagement and EVDetails functionality within the common UI.

• Test registration, duplicate IDs, updates, deletion, search and invalid input.

• Document EV model, endpoints and demonstration workflow.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

Mritunjay Thakur — Charging Stations

• Implement ChargingStation class and state transitions.

• Implement station database operations and endpoints.

• Implement availability, assignment and release logic.

• Complete ChargingStations and StationDetails functionality within existing UI.

• Test available, charging, maintenance and offline states.

• Test station assignment/release with sessions.

• Document station module and API contract.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

Amit Singh Kaintura — Battery + Battery Swap

• Implement Battery class and inventory states.

• Implement battery database operations.

• Implement swap workflow and REST APIs.

• Complete BatteryManagement and BatterySwap functionality.

• Handle READY, IN_USE, CHARGING and UNAVAILABLE states.

• Coordinate swap requests with Swap Queue.

• Test successful and unavailable-battery scenarios.

• Document battery/swap workflow.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

Abhishek Ghansala — UI + DSA + Final Integration

• Maintain React application shell, Material 3 theme, routing and reusable components.

• Continue and polish the existing frontend prototype and integrate module pages consistently.

• Implement simple frontend authentication state/route protection in coordination with the login API.

• Implement manual C++ Charging Queue, Swap Queue and Transaction Stack.

• Ensure Queue/Stack behavior is exposed correctly to backend APIs.

• Maintain final combined repository/build configuration.

• Compile everyone's code into one working project and resolve integration conflicts.

• Run full-system builds and integration tests.

• If another member's feature logic fails, return it to that member for correction rather than owning their feature code.

• Prepare the final integrated demonstration build.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

Agreev Singh — Transactions + Reports + Analytics

• Implement Transaction C++ model and transaction operations.

• Implement transaction/report/analytics API support.

• Complete Transactions page and LIFO visualization using backend data.

• Build Reports and Analytics using existing MUI/Recharts structure.

• Calculate/display session, revenue, energy, waiting-time and utilization statistics.

• Test transaction generation after charging and swap completion.

• Document reporting logic and demonstration scenarios.

Equal contribution: this module includes C++/backend work, frontend work, testing, documentation and presentation preparation. Ownership differs, but expected overall effort is equal across all six members.

16. Final Integration & Compilation

Each member owns the quality of their module. They must test it before submission. Abhishek owns the final combined build and repository integration. Integration is not a replacement for module ownership.

Workflow: member implements → member unit-tests → member submits branch/PR → compile/interface check → Abhishek integrates → full build → integration test → failing module returns to its owner → corrected module is re-integrated.

All members commit their work to the shared GitHub repository: **https://github.com/abhighansala/SmartEvHub.git**. The project should be integrated continuously instead of combining all modules immediately before the final demonstration.

17. Development Phases

Phase 1 — Requirements — Freeze UI pages, class responsibilities, database tables, authentication flow, API names and DSA interfaces.

Phase 2 — Frontend prototype — Complete current React UI with mock operational data; verify routes and interactions.

Phase 3 — C++ OOP — Implement and unit-test domain classes including User.

Phase 4 — DSA — Implement Queue and Stack in C++ and test boundary cases.

Phase 5 — SQLite — Create schema, CRUD operations and persistence tests.

Phase 6 — Authentication — Connect Login.tsx to simple email/password backend validation.

Phase 7 — Crow — Expose module functionality through REST endpoints.

Phase 8 — Integration — Replace mock operations with API calls and connect React to C++.

Phase 9 — System testing — Run complete charging, queue, swap, session, transaction and login scenarios.

Phase 10 — Finalization — Documentation, screenshots, demo script, viva preparation and final build.

18. Testing Strategy

Unit: each member tests their C++ classes, database functions and APIs. Queue/Stack test empty, normal and boundary cases.

Authentication: valid email/password, wrong password, unknown email, empty fields and logout.

Integration: verify React → API → C++ → SQLite for every module.

Scenario: fill stations → queue another EV → release station → dequeue → start session → complete session → generate transaction.

Persistence: modify data, restart backend, verify records remain.

UI: navigation, forms, validation, loading/error/empty states, responsive behavior and dark mode.

19. Validation & Error Handling

The backend validates required fields, unique IDs, numeric ranges, entity existence and legal state transitions. Frontend validation improves user experience but does not replace backend validation. Examples: charge must be 0–100%; an unavailable station cannot be assigned; a swap cannot complete without a READY battery; empty Queue cannot be dequeued; empty Stack cannot be popped; invalid login credentials must be rejected.

20. Git & Repository Rules

All members use the shared repository: **https://github.com/abhighansala/SmartEvHub.git**. Feature branches and pull requests are recommended. Main should remain buildable. Use meaningful commits, pull before work, avoid overwriting another member's code, and test before integration. Recommended structure: frontend/ for React, backend/ for C++/Crow, database/ for schema and docs/ for documentation.

Each member should make commits that clearly show their contribution. Before final submission, the team should verify that the repository contains the complete buildable project and documentation.

21. Final Demonstration

1) Open Login and authenticate using a valid email/password. 2) Show Dashboard. 3) Register/select EVs. 4) Fill charging stations. 5) Put another EV into Charging Queue. 6) Demonstrate FRONT/REAR FIFO. 7) Complete a session. 8) Show the next queued EV being processed. 9) Perform a battery swap and demonstrate Swap Queue if inventory is exhausted. 10) Complete an operation and show the transaction at the TOP of the Stack. 11) Open Reports/Analytics. 12) Restart the backend and

demonstrate database persistence.

22. Definition of Done

• Simple email/password login works through the C++ backend and SQLite.

• All specified React routes work and the Material 3 UI remains consistent.

• Mock operational state is replaced by real API calls for final features.

• C++ OOP classes compile and are tested.

• Charging Queue and Swap Queue are manually implemented in C++.

• Transaction Stack is manually implemented in C++.

• Crow APIs work for major modules.

• SQLite stores and retrieves persistent data.

• Charging, swapping, queuing, sessions and transactions work end-to-end.

• Reports and analytics reflect backend/database data.

• All six members' modules compile in one repository.

• Each member can explain their module, data flow, OOP/DSA use and testing.

• Final demonstration completes without manual database editing.

23. Learning Outcomes

The team will demonstrate practical C++ OOP, manual Queue and Stack implementation, SQLite/SQL, REST APIs, simple email/password authentication, React component architecture, state management, HTTP/JSON communication, modular software design, Git collaboration, testing and end-to-end integration.

24. Final Project Statement

Smart EV Hub is a web-based EV charging and battery-swap management simulation combining a React Material 3 frontend, C++ OOP backend, manually implemented Queue and Stack data structures, Crow REST APIs and SQLite persistence. Its central problem is resource management under limited charging and battery capacity. The project demonstrates how real-world operations can be modeled with OOP and DSA, persisted in a database, exposed through APIs and presented through a usable web interface.